JavaScript Moderno: La Guía de Supervivencia

Destructuring, **Colecciones** y **Asincronía** explicados de forma práctica para tu día a día en el front-end.

NotebookLM

El Mapa de Ruta

Acto 1: Dominando los Datos

- Destructuring (Objetos y Arrays)
- Colecciones Eficientes (Set y Map)

Acto 2: Dominando el Tiempo

- Promesas
- Async / Await

> Hoy aprenderemos a escribir código más limpio, predecible y profesional.

NotebookLM

Destructuring de Objetos

```
const usuario = {
  nombre: 'Ana',
  rol: 'Frontend',
  edad: 25
};

// Desempacando las propiedades
const { nombre, rol } = usuario;

console.log(nombre); // 'Ana'
console.log(rol);    // 'Frontend'
```

Definición

Sintaxis especial para 'desempacar' propiedades de un objeto y guardarlas en variables distintas en una sola línea.

Explicación Breve

En lugar de escribir usuario.nombre y usuario.rol repetidamente, creamos las variables nombre y rol directamente desde el objeto.

NotebookLM

Destructuring: Renombrando Variables

```
const producto = {
  id: 101,
  titulo: 'Teclado Mecánico'
};

// Renombrando 'titulo' a 'nombreProducto'
const { titulo: nombreProducto, id } = producto;

console.log(nombreProducto); // 'Teclado Mecánico'
// console.log(titulo);      // Daría un error
```

Definición
Si la propiedad original tiene un nombre genérico o repetido, puedes asignarle un nuevo nombre al extraerla.

Explicación Breve
Usamos los dos puntos (:) para decirle a JavaScript: “toma esta propiedad y guárdala bajo este nuevo nombre de variable”.

NotebookLM

Destructuring de Arrays

```
const colores = ['rojo', 'verde', 'azul'];

// Extrayendo por posición
              0              1
const [colorPrincipal, colorSecundario] = colores;

console.log(colorPrincipal);   // 'rojo'
console.log(colorSecundario);  // 'verde'
```

Definición

Funciona igual que en los objetos, pero la extracción se basa estrictamente en la posición (índice) del elemento, no en su nombre.

Explicación Breve

La primera variable toma el elemento en la posición 0, la segunda toma el de la posición 1. El nombre que le des a la variable no importa, solo su orden.

NotebookLM

Colecciones: Set

```
const visitas = new Set();

visitas.add('usuario_1');
visitas.add('usuario_2');
visitas.add('usuario_1'); // Duplicado, se ignora

console.log(visitas.size); // 2
```

Definición

Un Set es una colección de valores donde no se permiten duplicados. Cada valor dentro es estrictamente único.

Explicación Breve

Usamos **.add()** para insertar valores. Si intentamos agregar un valor que ya existe, el Set lo ignora silenciosamente. Usamos **.size** (no .length) para saber cuántos elementos únicos hay.

NotebookLM

Colecciones: Map

```
const configuracion = new Map();

configuracion.set('tema', 'oscuro');  // Clave String
configuracion.set(404, 'Not Found');  // Clave Number

console.log(configuracion.get(404));    // 'Not Found'
console.log(configuracion.has('tema')); // true
```

Definición

Un Map es una colección de pares clave-valor (como un objeto), pero con una gran ventaja: las claves pueden ser de cualquier tipo de dato.

Explicación Breve

Usamos .set(clave, valor) para guardar datos y .get(clave) para leerlos. El método .has() permite verificar rápidamente si una clave existe.

NotebookLM

El Problema del Tiempo

En el desarrollo front-end, muchas operaciones no ocurren de inmediato. Pedir datos a una API, encender la cámara del usuario o leer un archivo toma tiempo.

> **El Reto:** ¿Cómo le decimos a nuestro código "Espera a que esto termine antes de continuar"?

La Solución: Promesas.

NotebookLM

Promesas: El Futuro de tus Datos

```
const pedirPizza = new Promise((resolve, reject) => {

  // Simulamos que la preparación toma 2 segundos
  setTimeout(() => {
    resolve('🍕 ¡Pizza lista!');
  }, 2000);
});
```

Definición

Una Promesa representa el resultado de una operación que aún no termina. Es como el localizador de un restaurante: 'Prometen' avisarte cuando tu comida esté lista.

Explicación Breve

Creamos una promesa con `new Promise`. Recibe una función con `resolve` (para reportar éxito) y `reject` (para reportar una falla).

NotebookLM

Promesas: Suscribiéndose al Éxito

```
1  console.log('1. Haciendo el pedido...');

   pedirPizza.then((mensaje) => {
     // Esto se ejecuta en el futuro
3    console.log('3.', mensaje); // '🍕 ¡Pizza lista!'
   });

2  console.log('2. Viendo el celular mientras espero...');
```

Definición

Para 'consumir' el resultado de una Promesa una vez que se resuelve, nos suscribimos a ella usando el método .then().

Explicación Breve

El código no se detiene. JavaScript sigue ejecutando las siguientes líneas y ejecutará el contenido del .then() solo cuando la promesa llame a resolve().

NotebookLM

Async / Await: El Estándar Moderno

Una sintaxis especial que hace que el código asíncrono se lea como si fuera código síncrono.

La palabra **async** convierte una función regular en una que siempre devuelve una Promesa.

La palabra **await** sólo se puede usar dentro de funciones async. Literalmente pausa la ejecución de esa función hasta que la promesa entregue su resultado.

NotebookLM

Async / Await en Acción

```
// Usando la promesa 'pedirPizza' anterior

async function cenar() {
  console.log('Haciendo el pedido...');

  // await 'pausa' la función aquí
  const mensaje = await pedirPizza;

  console.log(mensaje); // '🍕 ¡Pizza lista!'
  console.log('¡A comer!');
}

cenar();
```

En lugar de pasar un callback a `.then()`, asignamos el resultado de `await pedirPizza` **directamente** a una variable.

El código se lee de arriba hacia abajo, de forma natural.

NotebookLM

La Evolución: .then() vs async/await

El Pasado: Tradicional

```
function cargarPerfil() {
  obtenerUsuario()
    .then(usuario => obtenerPermisos(usuario.id))
    .then(permisos => console.log(permisos));
}
```

El Presente: Moderno

```
async function cargarPerfil() {
  const usuario = await obtenerUsuario();
  const permisos = await obtenerPermisos(usuario.id);
  console.log(permisos);
}
```

> **async/await** funciona sobre las Promesas. Simplemente nos da una forma mucho más limpia de escribirlas, evitando el infierno de la anidación (callback hell).

NotebookLM

Cheat Sheet Visual: Guarda esta diapositiva

Extraer Datos

```
{ a, b } = obj
[ a, b ] = arr
```

Almacenar Datos

Set: Valores únicos (add, size)
Map: Claves flexibles (set, get)

El Futuro

```
new Promise((resolve, reject))
Uso: .then(resultado => ...)
```

Esperar el Futuro

```
async function()
  const res = await promesa;
```

NotebookLM

Menos código, más lógica.

```
status === 'ready_to_code'
```

Abre tu editor, declara un Objeto, crea una Promesa
y transpórtala al futuro con Async/Await.

¡A practicar!

NotebookLM